Colônia de Formigas Distribuída para o Caixeiro-Viajante com Eleição de Líder, Relógio de Lamport e Exclusão Mútua

Pedro Henrique Vilaça Valverde[1], **Marcus Prado Silva**[1],
Gabriel Batista de Almeida[1], **Eduardo Alves Salgado Lisboa Oliveira**[1]

[1]Instituto de Ciências Exatas e Informática (ICEx) – PUC Minas
Belo Horizonte – MG – Brasil

Resumo. ***[PENDENTE: resumo em português, até 8 linhas (5 frases: problema, abordagem, coordenação, números, conclusão)]***

Abstract. ***[PENDENTE: abstract em inglês, até 8 linhas]***

Palavras-chave: computação distribuída; colônia de formigas; eleição de líder; exclusão mútua; relógio lógico

1. Introdução

[PENDENTE: problema: TSP, berlin52 e por que heurística]

[PENDENTE: por que distribuir: formigas independentes, único acoplamento é a matriz]

[PENDENTE: o que foi construído: Java 21 + gRPC, líder eleito, matriz replicada]

[PENDENTE: 4 requisitos atendidos (o enunciado exige 2) e a organização do artigo]

2. Arquitetura do Sistema

[PENDENTE: visão geral: N nós simétricos, mesmo binário, papel definido pela eleição]

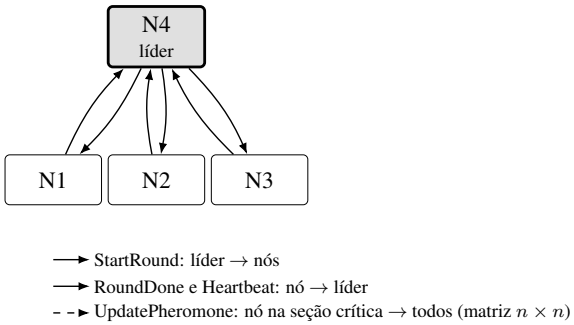


Figura 1. Entidades e mensagens. Todos os nós são trabalhadores; o líder (maior id vivo) coordena as rodadas. A matriz de feromônio é replicada e só muda dentro da seção crítica.

2.1. Entidades e papéis

[PENDENTE: líder (RoundCoordinator), trabalhador (RoundWorker), detector (LeaderMonitor) e módulos de suporte]

Mensagem	Direção	Papel
`Heartbeat`	nó → líder	detector de falha e handshake de configuração
`Election` / `Coordinator`	todos → todos	eleição (Bully)
`RequestCS` / `ReplyCS`	nó ↔ nó	Ricart-Agrawala (com `request_ts`)
`StartRound`	líder → todos	início de rodada
`RoundDone`	nó → coordenador	melhor rota local da rodada
`UpdatePheromone`	nó → todos	matriz $n \times n$ + melhor rota (síncrono)

Tabela 1. Mensagens do serviço `NodeService` (gRPC). Toda mensagem carrega `sender_id` e um carimbo de Lamport no cabeçalho, além do token do cluster.

2.2. Mensagens

[PENDENTE: por que um serviço único, e o que vai no cabeçalho (sender_id, lamport, token)]

2.3. Uma rodada

[PENDENTE: formigas (ITERATIONS_PER_ROUND) → seção crítica (atualiza + propaga) → RoundDone para quem coordenou]

3. Requisitos Implementados

3.1. RPC – gRPC

[PENDENTE: 8 RPCs unários; deadlines por ambiente; token no interceptor + sender_id em PEERS; 64 MiB]

3.2. Eleição de líder – Bully

[PENDENTE: 3 passos; maior id vence; contestação de COORDINATOR menor; detector separado, 5 heartbeats; T̃0 s]

3.3. Relógio lógico – Lamport

[PENDENTE: tick no envio, max+1 no recebimento; ordena pedidos de SC e numera os logs; por que não relógio físico]

3.4. Exclusão mútua – Ricart-Agrawala

[PENDENTE: RELEASED/WANTED/HELD; prioridade (ts, id); REPLY carrega request_ts; 2(N-1); reenvio a cada 500 ms; falha = permissão]

4. Resultados

[PENDENTE: o que a tabela mostra: réplicas convergem, 7542 = ótimo, custo por rodada, failover]

[PENDENTE: o failover de 15,6 s → 0,56 s (F14): medir de ponta a ponta expôs um custo que o teste unitário não via]

[PENDENTE: ensaio em 2 máquinas físicas (se já feito; senão apagar este parágrafo)]

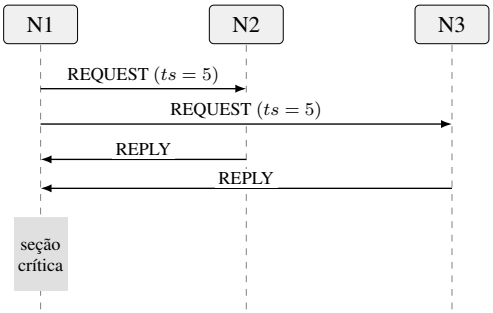


Figura 2. Ricart-Agrawala com três participantes: N1 só entra na seção crítica depois de receber as duas permissões. O par (ts, id) decide quem espera: um pedido mais novo pode ser adiado e respondido só na saída da seção crítica.

Medida	Valor observado
Testes automatizados	43 em 12 classes, 0 falhas
Rodada (berlin52, 40 formigas/nó, I=1000)	≈0,94 s (mediana; 0,84–1,36 s)
Melhor rota em berlin52 (20–30 rodadas)	7542 (ótimo conhecido)
Failover do líder (4 nós, perfil LAN)	≈10 s até `Novo líder: N3`
1ª rodada após a queda	0,56 s (era 15,6 s antes do F14)
Execução longa / queda de seguidor	4000 rodadas sem timeout; 600 concluídas

Tabela 2. Medidas do sistema. Exceto a execução longa (auditoria de 20/10), os tempos foram obtidos em uma única máquina com 4 contêineres; I é `ITERATIONS_PER_ROUND`. Não medimos speedup de 1×N nós.

5. Desafios e Problemas Encontrados

[PENDENTE: lento não é morto: falha por timeout conta como permissão (F1, não corrigido, decisão de projeto)]

[PENDENTE: peer morre depois do REQUEST: acquire() preso; reenvio a cada 500 ms (F2/F3)]

[PENDENTE: configuração divergente = split-brain silencioso; handshake encerra com código 2 (F9)]

6. Papel de Cada Aluno

Aluno	Módulo	O que fez
Pedro Henrique Vilaça Valverde	gRPC, Docker, LAN	IPs fixos e timeouts de LAN, handshake de token, TSPLIB
Marcus Prado Silva	Ricart-Agrawala + documentação	implementação e testes de integração, docs por classe/algoritmo, script `tp`
Gabriel Batista de Almeida	**[PENDENTE: módulo]**	**[PENDENTE: 2 a 3 itens]**
Eduardo Alves Salgado Lisboa Oliveira	**[PENDENTE: módulo]**	**[PENDENTE: 2 a 3 itens]**

Tabela 3. Divisão de trabalho. Descrever o que a pessoa realmente fez (projetou, implementou, testou, corrigiu, escreveu), não apenas o módulo.

Referências

[1] LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, v. 21, n. 7, p. 558–565, 1978.

[2] RICART, G.; AGRAWALA, A. K. An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, v. 24, n. 1, p. 9–17, 1981.

[3] GARCIA-MOLINA, H. Elections in a distributed computing system. *IEEE Transactions on Computers*, v. C-31, n. 1, p. 48–59, 1982.

[4] DORIGO, M.; GAMBARDELLA, L. M. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, v. 1, n. 1, p. 53–66, 1997.

[5] REINELT, G. TSPLIB – a traveling salesman problem library. *ORSA Journal on Computing*, v. 3, n. 4, p. 376–384, 1991.

[6] GRPC. *gRPC documentation*. Disponível em: `https://grpc.io/docs/`. Acesso em: out. 2026.